

```
=====
ShinySwarm R Package - Build Session Transcript
=====
```

```
Project : Collaborative-RShiny (ShinySwarm) Master's thesis, UvA
Author  : Iva Gunjaca
Agent   : Ona
Topic   : Building, reviewing, and hardening the 'shinyswarm' R package prototype
          from the thesis future-work notes.
```

This is a human-readable summary transcript of the working session. It records the conversation flow, decisions, and outcomes. For HOW the library was built in reproducible detail, see k6/results/kafka/thesis-notes.tex (unchanged) and the package source under shinyswarm/.

```
-----
1. REQUEST: Build a working prototype of the future-work R library
-----
```

```
User: "look at my docs there you will find future work R library can you make a
      working prototype? install whatever you need to test it"
```

The future-work spec lived in k6/results/kafka/thesis-notes.tex, section "Future Work: Reusable R Package for State Sharing". It described a transport-agnostic package named 'shinyswarm' with:

- capture_state(input, ...) -> serialise inputs + reactives to JSON
- restore_state(session, json, ...) -> push values back
- typed wrappers state_file() and state_rds()

(The docs were initially not pushed; user pushed and agent pulled to read them.)

```
-----
2. ENVIRONMENT SETUP
-----
```

- Installed R 4.3.3 (Ubuntu 24.04, r-base-core).
- System deps: libcurl4-openssl-dev, libssl-dev, libxml2-dev.
- R packages via Posit binary repo (User-Agent header needed for binaries):
 jsonlite, base64enc, shiny, testthat, later roxygen2.
- Package installed to ~/Rlib (no permission for site-library).

```
-----
3. INITIAL IMPLEMENTATION
-----
```

```
Created package skeleton at shinyswarm/:
DESCRIPTION, NAMESPACE, LICENSE
R/capture_state.R, R/restore_state.R, R/wrappers.R, R/utils.R
inst/examples/calculator/app.R
tests/testthat/*
```

Key design points implemented:

- Transport-agnostic: functions emit/consume JSON only.
- Reactives registered by name through ... (R cannot auto-discover them).
- Typed dispatch in .capture_one / .restore_one.
- digits = NA in toJSON to preserve full numeric precision (critical for exact reproducibility of non-deterministic results).

First test run: 1 failure - JSON rounded a Monte Carlo value to 4 decimals.
Fix: digits = NA. All tests passed afterward (33 checks).
Calculator demo booted and served HTTP 200 locally.

```
-----
4. CODE REVIEW #1 (skill: "Code Review like Bart")
-----
```

```
Findings (most important = security, because cross-user restore = untrusted input):
[High]      Path traversal via attacker-controlled filename (PoC confirmed:
            filename "../ESCAPED.txt" wrote outside target dir).
```

[Med/High] unserialize() on untrusted checkpoint data (code-exec risk).
[Medium] reactiveValues restore was additive (union), not exact -> cross-contamination risk; also untested.
[Low] Undeclared stats:: dependency.
[Minor] null round-trip behaviour; friendlier fromJSON error; is.function branch can misclassify a plain function.

Also noted: no man/ pages, placeholder email, hand-written NAMESPACE vs roxygen.

5. FIX ROUND #1 (user: "yes do this")

Explanations given to user:

- reactiveValues "additive vs exact": restore only wrote saved keys, never cleared stale keys present in receiving session -> result was the union.
- is.function misclassification: a reactiveVal IS a function (read via rv()), so a literal function passed as data is treated as a reactiveVal and called.

Implemented:

- Path traversal: .safe_filename() rejects separators, drive letters, ~, ..
- unserialize gated behind allow_rds = FALSE (default), documented in a @section Security block.
- max_size guard (default 50 MB) on capture and restore.
- gzip compression (compress = TRUE) for file/rds payloads.
- Exact reactiveValues restore: clear stale keys (set to NULL, since Shiny cannot truly delete a key) + new test.
- Friendly fromJSON error wrapping.
- Dropped stats:: (used names<-).
- DESCRIPTION email -> gunjaca.iva@gmail.com.
- roxygen-generated NAMESPACE + man/ pages.
- Found subtle Rd bug: literal "%" in "~33%" is an Rd comment char that broke capture_state.Rd; replaced with "percent".

New tests: test-security.R, test-reactivevalues.R, test-edgecases.R.

Result: R CMD check Status: OK (no errors/warnings/notes); all tests green.

6. EXAMPLE FILES (user: "do you have example files i could run")

Added inst/examples/demo-roundtrip.R - a headless script (no server/browser) demonstrating: basic round-trip, non-deterministic preservation, file embedding, and path-traversal rejection. Verified it runs.

(User instruction: "dont serve it anywhere i will download it" - no servers served thereafter.)

7. CODE REVIEW #2

Confirmed all prior fixes resolved. New findings:

- [Medium] Decompression bomb: size was checked AFTER full memDecompress allocation -> small payload could expand to many GB before the guard fired (PoC: 20 KB -> 20 MB).
- [Low] Opaque error on compression mismatch.
- [Low] rds/file restore silently no-ops if no setter supplied.
- [Low] demo run instruction assumed installed package.
- [Housekeep] No .gitignore for R build artifacts.

8. FIX ROUND #2 (user: "both" - fix decompression bomb + write README + thesis tex)

Decompression bomb fix:

- Rewrote .gzip_raw/.gunzip_raw to use gzfile connections (true gzip framing; memCompress(type="gzip") actually emits zlib, which gzfile passes through unchanged - that mismatch was caught during testing).
- .gunzip_raw now streams in 1 MB chunks and aborts as soon as the running total exceeds max_size, so peak allocation is bounded.

- Verified: a 508 KB payload claiming 500 MB aborts in ~18 ms without allocating the full output.
- Added regression tests (bomb rejected; normal compressed payload still ok).
- Made demo self-sourcing for source checkouts.

Result: R CMD check Status: OK again; full suite green.

Deliverables written:

- shiny swarm/README.md (install, usage, wrappers, transport, security table, limitations, examples, testing, status).
- docs/thesis/shiny swarm_library.tex (standalone thesis section: overview, motivation, design, reproducibility guarantee, security, testing, limitations, contribution). thesis-notes.tex left UNTOUCHED per user request.

9. THESIS GUIDANCE (user asked: can this go in my thesis if documented well?)

Agent advice summary:

- Yes - frame as Future Work / validated proof-of-concept, NOT a core benchmarked contribution.
- Strengths to lean on: it's tested, R CMD check clean, limitations documented.
- Frame the security findings as a research observation: full-state sharing introduces a trust boundary that input-only sharing does not - ties to RQ5.
- Don't oversell LOC (~250 lines); its value is what it demonstrates.
- Suggested one-paragraph framing was provided (now expanded in the .tex).

FINAL STATE OF DELIVERABLES

Package (shiny swarm/):

DESCRIPTION, NAMESPACE, LICENSE, README.md
R/capture_state.R, R/restore_state.R, R/wrappers.R, R/utils.R
man/*.Rd (5 files, roxygen-generated)
inst/examples/calculator/app.R
inst/examples/demo-roundtrip.R
tests/testthat/{helper-fakes, test-capture, test-restore, test-wrappers,
test-edgecases, test-reactivevalues, test-security, test-integration-shiny}.R

Docs:

docs/thesis/shiny swarm_library.tex (new, thesis-ready)
docs/shiny swarm-build-session.txt (this file)
k6/results/kafka/thesis-notes.tex (UNCHANGED - reproducibility record)

Verification:

- R CMD check: Status: OK (0 errors, 0 warnings, 0 notes)
- Full testthat suite passing (capture, restore, wrappers, edgecases, reactivevalues, security, integration-shiny)
- Calculator demo serves HTTP 200 locally (not exposed)
- Decompression-bomb guard verified to abort early with bounded memory

Nothing committed to git unless explicitly requested.

=====